

WAPAPP 2.0

Enterprise Architecture Proposal

Document Type:	Architecture Design & Migration Plan
Project:	WapApp 2.0 (Legacy Tittu → New Platform)
Prepared For:	Management Review
Date:	27 July 2026
Status:	Planning Phase — Backend development not yet started



1. Executive Summary

We are rebuilding **Tittu** (WhatsApp Business API SaaS platform) as **WapApp 2.0** — a modern, enterprise-grade platform.

The legacy system currently serves **30 active customers** sending approximately **1,00,000 messages per week**. Its architecture has reached scalability and reliability limits and needs to be replaced.

The new version is built on **Laravel 13** with a clean, domain-driven architecture. The UI is approximately **85% complete** (233 screens, Figma-driven). Backend development is greenfield, giving us the opportunity to design the system correctly from the ground up.

Key Goals

- Master DB + Tenant DB architecture for true data isolation
- Enterprise-level performance capable of handling 10x current load
- Zero data loss and zero downtime during normal operations
- Proper error handling, monitoring, and alerting
- Complete redesign of the webhook processing pipeline
- Module-by-module migration from legacy without disrupting existing customers

2. Current State Assessment

2.1 Legacy System (wapdev.tittu.in)

Aspect	Current State	Risk
Architecture	Single MySQL DB, row-level tenancy	High
Framework	Laravel 8, forked from Acelle Mail	Medium
Queue System	Database-driven queues	High
Webhook Processing	Staging table → cron → delete before process	Critical
Code Quality	God classes (2,300+ lines), 99 controllers	High
Scheduling	10+ cron jobs running every minute	High
Monitoring	Shell scripts for production fixes	Critical
AI Service	Separate Python FastAPI (good, to be retained)	Low

Critical Legacy Problems

1. **Webhook staging table** — Messages inserted into a webhooks table, then deleted BEFORE processing. If processing fails, data is permanently lost.
2. **Database queue** — Queue jobs stored in MySQL cause DB overload during campaign bursts.
3. **Cron-heavy processing** — Automation, campaigns, webhooks, and integrations all poll via cron every minute.
4. **No tenant isolation** — All 30 customers share one database; one customer's heavy campaign affects all others.
5. **Mixed concerns** — Email marketing (Acelle) and WhatsApp logic coexist in the same codebase.
6. **No structured error handling** — Errors are inconsistent, making debugging and customer support difficult.

2.2 New System (wapapp-2.0)

Aspect	Current State
Framework	Laravel 13, PHP 8.3
Frontend	Blade + Tailwind CSS 4 (~233 views, 106 routes)
UI Progress	~85% complete (Figma-driven)
Backend Progress	~0% (greenfield — default Laravel skeleton only)
Database	SQLite (development only)
Auth	UI only — no backend wired
API / Integrations	Not started

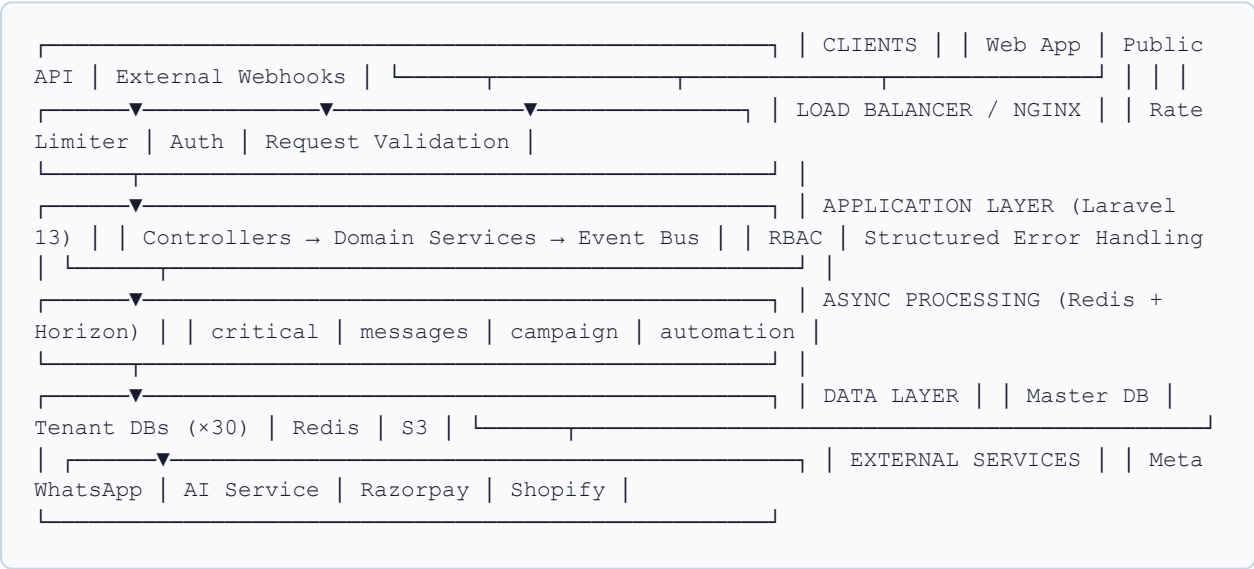
Advantage: Complete UI prototype and a clean slate for backend architecture.

3. Production Requirements

Requirement	Detail
Active customers at launch	30 (existing, must migrate)
Message volume	~1,00,000 messages/week (~14,300/day)
Peak load	Campaign bursts of 5,000–10,000 messages in 5 min
Growth target	Architecture must support 300+ customers (10x)
Uptime target	99.9% (~8.7 hours max downtime/year)
Delivery rate	> 95%
Webhook response to Meta	< 100ms (Meta requires 200 within 20 seconds)
Data loss tolerance	Zero — all events logged immutably

4. Proposed Architecture

4.1 High-Level System Design



4.2 Database Architecture — Master DB + Tenant DB

Master Database (Platform Level)

Table	Purpose
tenants	Tenant identity, plan, status, DB config
domains	Custom domain mapping per tenant
plans, subscriptions, invoices	Billing and subscription management
admins	Super admin users
global_settings	Platform-wide configuration
tenant_provisioning_logs	Audit trail for tenant creation
platform_webhook_events	Meta app-level webhook log (immutable)

Tenant Database (Per Customer — 30 separate databases)

Table Group	Tables
Users & Access	users, team_members, roles, permissions
Messaging	contacts, conversations, messages, inbox_assignments
Campaigns	templates, campaigns, campaign_runs, campaign_logs
Automation	automations, automation_runs, chatbot_flows
Audience	lists, segments, segment_rules
Webhooks	webhook_events, webhook_subscriptions, webhook_deliveries
Commerce	products, orders, payments
Integrations	integrations (Shopify, Calendly, Zoho tokens)
Billing	wallet_transactions

Why database-per-tenant?

- True data isolation — one customer's load cannot affect others
- Independent backup and restore per customer
- Compliance-ready (data residency per tenant in future)
- Clean migration — migrate one customer at a time

4.3 Webhook Architecture — Complete Redesign (Most Critical Change)

LEGACY FLOW (Problematic):

Meta POST → Insert into webhooks table → Cron polls every minute → Delete row BEFORE processing → Process → If fails, data is LOST

NEW FLOW (Reliable):

Meta POST → Verify HMAC signature → Check idempotency (duplicate? skip) → Append to webhook_events (immutable) → Dispatch to Redis queue → Return 200 to Meta (< 100ms) → Worker processes async → On failure: retry 3x → dead letter queue → alert team

4.4 Queue Architecture (Redis + Horizon)

Queue	Purpose	Workers	Timeout	Retries
critical	Inbound webhooks, real-time	5	30s	5
messages	Outbound message sending	4	60s	3
status	Delivery/read status updates	3	30s	3
campaign	Campaign batch sending	3	120s	3
automation	Chatbot, drip triggers	2	60s	3
default	Notifications, emails	2	60s	3
low	Reports, exports, cleanup	1	300s	2
ai	AI chatbot requests	2	60s	3
provisioning	Tenant DB creation	1	300s	2

4.5 Code Structure (Domain-Driven Design)

app/		
├─ Domains/		
└─ Auth/	Login, Register, 2FA	
└─ Tenant/	Provisioning, DB switching	
└─ Messaging/	Inbox, conversations, send	
└─ Campaign/	Create, schedule, send, track	
└─ Automation/	Flows, drip, chatbot	
└─ Webhook/	Inbound + Outbound	
└─ AI/	Chatbot RAG, key management	
├─ Shared/		
└─ Contracts/	Interfaces	
└─ DTOs/	Data transfer objects	
└─ Exceptions/	Domain-specific exceptions	

5. Error Handling & Reliability

5.1 Structured Error Response

```
{
  "success": false,
  "error": {
    "code": "CAMP_AUDIENCE_EMPTY",
    "message": "Campaign cannot be sent without an audience.",
    "details": { "campaign_id": "camp_123", "audience_count": 0 },
    "request_id": "req_abcl23",
    "timestamp": "2026-07-27T10:30:00Z"
  }
}
```

5.2 Error Code Categories

Prefix	Domain	Examples
AUTH_	Authentication	Token expired, invalid credentials
TENANT_	Tenancy	Suspended, limit exceeded
MSG_	Messaging	Send failed, template not approved
CAMP_	Campaigns	Audience empty, already running
WALLET_	Billing	Insufficient balance
WEBHOOK_	Webhooks	Invalid signature, delivery failed
SYS_	System	Service unavailable, rate limited

5.3 Reliability Mechanisms

Mechanism	Application
Idempotency	Duplicate webhooks, sends, debits = no-ops
Retry with backoff	3 retries: 10s → 60s → 300s, then dead letter
Circuit breaker	Meta API down → pause sending, queue, auto-resume
Append-only logs	Webhook events never deleted — replay possible
Dead letter queue	Failed jobs after retries → manual review + alert
Health checks	/up (basic) + /health/deep (DB, Redis, queue, Meta)

5.4 Alert Severity

Level	Trigger	Response
P1 Critical	Message sending down, webhook failing	< 15 min
P2 High	Queue backlog > 1,000, Meta circuit open	< 1 hour
P3 Medium	Failed job rate > 5%, slow queries	< 4 hours
P4 Low	Single tenant error, non-critical	Next day

6. Security

Area	Implementation
Authentication	Laravel Sanctum (SPA sessions + API tokens)
Authorization	Spatie Permission (RBAC: owner, admin, agent)
Tenant isolation	Middleware enforces DB switch before any query
Two-factor auth	TOTP for admin/owner accounts
Webhook verification	HMAC-SHA256 on all inbound and outbound
Data encryption	Tenant DB credentials encrypted at rest
Audit logging	Per-tenant audit trail
Rate limiting	Per-tenant, per-IP, per-API-key

7. Performance & Caching

Component	Strategy
Queue driver	Redis (10x faster than legacy database queue)
Cache	Redis with tenant-scoped keys
Search	Meilisearch (replaces MySQL LIKE queries)
Media storage	S3 + CDN (replaces local storage)
Real-time inbox	Laravel Reverb WebSockets (replaces polling)
Campaign sending	Chunked batches of 50, parallel workers

8. Monitoring & Observability

Tool	Purpose	Environment
Laravel Horizon	Queue monitoring dashboard	Production
Sentry	Error tracking with full context	Production
Laravel Pulse	App metrics (requests, slow queries)	Production
Structured logs	tenant_id, request_id, user_id	All environments

9. Migration Strategy

Module-by-module migration. Legacy runs in parallel. Each customer cut over individually. No big-bang.

Phase	Module	Duration	Depends On
1	Foundation: Master DB, tenancy, auth, Redis, errors	Week 1–2	None
2	Core Messaging: webhooks, send, inbox	Week 3–4	Phase 1
3	Campaigns + Templates: create, send, Meta sync	Week 5–6	Phase 2
4	Automation: chatbot, drip, AI integration	Week 7–8	Phase 2
5	Commerce + Integrations: Shopify, Calendly, etc.	Week 9–10	Phase 1
6	Billing + Admin: subscriptions, wallet, panel	Week 11–12	Phase 1
7	Data Migration: legacy → tenant DBs, per-customer	Week 13+	All

10. Tech Stack

Layer	Technology	Rationale
Backend	Laravel 13, PHP 8.3	Latest stable, already chosen
Frontend	Blade + Tailwind CSS 4	UI already built
Multi-tenancy	stanc/tenancy	Database-per-tenant, proven
Queue	Redis + Laravel Horizon	10x faster than DB queue
Database	MySQL 8	Master + per-tenant DBs
Search	Meilisearch	Fast contact/message search
Storage	AWS S3 + CloudFront	Scalable media and exports
Real-time	Laravel Reverb	WebSocket inbox updates
AI	Python FastAPI (separate)	RAG chatbot, retained from legacy
Monitoring	Sentry + Horizon + Pulse	Errors, queues, metrics
CI/CD	GitHub Actions	Automated test and deploy

11. Architecture Principles

1. Tenant isolation first — no query without tenant context
2. Events over cron — reactive processing, not polling
3. Append-only logs — webhook/event data never deleted
4. Idempotent everything — duplicates are no-ops, not errors
5. Thin controllers, fat services — testable business logic
6. Queue everything heavy — user response always < 200ms
7. API-first — UI consumes internal API
8. Backward-compatible migrations — zero-downtime deploys
9. Feature flags — gradual rollout, instant rollback
10. No god classes — 300-line file limit, domain boundaries

12. SLA Targets

Metric	Target
System uptime	99.9%
Webhook response to Meta	< 100ms (p99)
Message send latency	< 2 seconds (p95)
Campaign throughput	50 messages/sec per number
Message delivery rate	> 95%
Job failure rate	< 0.1%
Data loss	0%

13. Risk Assessment

Risk	Impact	Mitigation
Migration data loss	Critical	Append-only logs, per-tenant rollback
Campaign burst overload	High	Separate queue, chunked send, circuit breaker
Meta API downtime	High	Circuit breaker, queue, auto-resume
Single tenant DB failure	Medium	One customer affected, independent backups
Legacy parallel run complexity	Medium	Module-by-module, not big-bang

14. Current Status & Next Steps

Area	Progress
UI (screens, nav)	~85% complete
Architecture design	Complete (this document)
Backend development	Not started
Legacy analysis	Complete

Recommended Next Steps

#	Action	Priority
1	Review and approve this architecture proposal	Immediate
2	Detailed DB schema design (Master + Tenant)	Week 1
3	Confirm module priority for implementation	Week 1
4	Infrastructure setup (Redis, MySQL, S3, staging)	Week 1
5	Phase 1: Foundation (tenancy, auth, queues)	Week 1–2
6	Phase 2: Core messaging (webhooks, send, inbox)	Week 3–4

Open Questions for Management

- Go-live target date — when should the first customer be migrated?
- Module priority — confirm inbox/messaging first, then campaigns?
- Infrastructure — AWS, DigitalOcean, or existing hosting?

- 4. Legacy parallel run — how long should both systems run?
- 5. Team size — how many developers on backend implementation?

15. Legacy vs New — Comparison

Aspect	Legacy (Tittu)	New (WapApp 2.0)
Framework	Laravel 8 (Acelle fork)	Laravel 13 (greenfield)
Tenancy	Single DB, customer_id column	Master DB + Tenant DB per customer
Queue	Database driver	Redis + Horizon
Webhooks	Staging table → cron → delete	Verify → log → queue (append-only)
Campaigns	Cron every minute	Chunked jobs, dedicated queue
Automation	Cron 3x per minute	Event-driven
Error handling	Inconsistent	Structured JSON, error codes
Monitoring	Shell scripts	Horizon + Sentry + Pulse
Real-time inbox	Polling / FCM only	WebSockets (Reverb)
Deployment	Manual	CI/CD (GitHub Actions)
Testing	Minimal	70%+ coverage target

— End of Document —